



5-1. 리소스 효율적 AI 모델

목차

1. 파인튜닝 및 AI 모델 활용 (리소스 효율적 AI 모델)

1. 컴퓨터의 수 체계

1-1. 고정 소수점 수 체계

1-2. 부동소수점 수 체계

1-3. 연산과 복잡도

2. 모델 경량화와 효율적 파인 튜닝

2-1. 모델 경량화의 필요성

2-2. Quantization/Pruning/Distillation

2-3. LoRA / QLoRA

1. 파인튜닝 및 AI 모델 활용 (리소스 효율적 AI 모델)

학습 목표

- 컴퓨터 시스템에서 숫자를 표현하기 위해 사용하는 다양한 이진 수 체계의 특징을 이해합니다.
- 주어진 수 체계를 바탕으로 연산 및 메모리 사용량이 야기하는 복잡도 차이를 학습합니다.
- 파라미터 효율적 파인튜닝의 개념을 이해하고, 이로 인한 모델의 정확도-복잡고 변화를 이해합니다.
- 보다 다양한 최신 모델 경량화 기법의 특징을 이해합니다.

학습 시작(OverView)

- 컴퓨터는 숫자를 어떻게 이해할까?
 - 고정소수점/부동소수점 이진 수 체계 및 연산 과정
- 컴퓨터 내 연산은 얼마일까?
 - 연산/데이터 크기별 하드웨어 복잡도 및 에너지 소비
- 모델 압축 어디까지 가능한가?
 - Quantization, pruning 및 distillation
- 파인튜닝? 파라미터 효율적 튜닝?
 - 쉽게 적용가능한 파인튜닝의 효과 분석

1. 컴퓨터의 수 체계

1-1. 고정 소수점 수 체계

수 체계

- 10진수 : 10의 지수로 표현하기
- 2진수 : 2의 지수로 표현하기

2진수 → 10진수 변환

10진수 → 2진수 변환

2의 거듭제곱

- $2^{10} = 1 \text{ Kilo} = 1000$
- $2^{20} = 1 \text{ Mega} = 1 \text{ million}$
- $2^{30} = 1 \text{ Giga} = 1 \text{ billion}$
- 2^{24} 의 크기? 16million
- 32bit 변수가 표현할 수 있는 패턴의 가지수는?
 - $2^2 * 2^{30} = 4 \text{ billion}$

16진수의 등장

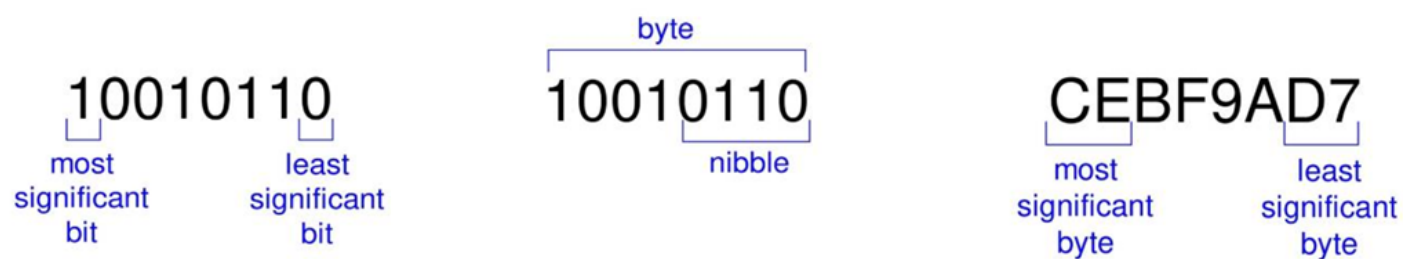
- 2진수는 너무 길다

16진수 → 2진수 변환

10진수 → 16진수 변환

Bits, Bytes, Nibbles

- MSB : 2진수에서 가장 왼쪽에 있는 비트, 가장 큰 자리 값을 가짐
- LSB : 2진수에서 가장 오른쪽에 있는 비트, 가장 작은 자리 값을 가짐
- Nibble : 4 bite(half byte)로 이뤄진 데이터 단위



이진수의 값과 범위

- 0과 자연수를 표현하는 N-자리 십진수
 - 가능한 표현의 수 ? 10^N
 - 범위? 0, 10^N-1
 - 3-digit 에서 10진수
 - $10^3 = 1000$ 까지 표현 $\Rightarrow [0, 999]$
- 0과 자연수를 표현하는 N 자리 2진수
 - 가능한 표현의 수? 2^N
 - 범위? 0, 2^N-1
 - 3digit 2진수 예시
 - $2^3 = 8$ 가지 표현 $\Rightarrow [0, 7]$

이진수의 덧셈

- bit 크기에 따라 잘린다.

$$\begin{array}{r} 1 \\ 1001 \\ + 0101 \\ \hline 1110 \end{array}$$

$$\begin{array}{r} 111 \\ 1011 \\ + 0110 \\ \hline 10001 \end{array}$$

Carry-out

부호 있는 수 체계

- 부호 - 크기 표현 방식 (signed-magnitude representation)
 - 초기에 도입되었다가 사라진 방식
- 2의 보수 표현 방식 (two's complement representation)

부호 크기 표현 방식

1 sign bit, N-1 magnitude bit

- 최상위 자리에 sign bit 할당 (부호 자리 , MSB)
 - 양수 → sign bit = 0
 - 음수 → sign bit = 1
- 4bit 부호 크기 표현 방식
 - ex) 6 = 0110, -6 = 1110
- N-bit 부호 크기 표현의 범위
 - [-(2^(N-1)-1), 2^(N-1) -1]

기존 덧셈에 문제가 생김

• -6 + 6 예시

$$\begin{array}{r} 1110 \\ + 0110 \\ \hline 10100 \end{array}$$

0을 표현하는 방식에 두가지가 있음 (+0, -0)

- 1000
- 0000



부호-크기 체계는 단점으로 인해 폐기되고 2의 보수를 이용하게 된다.

2의 보수 표현 방식

- 최상위 자리가 -2^(N-1) 값을 표현

$$A = a_{n-1}(-2^{n-1}) + \sum_{i=0}^{n-2} a_i 2^i$$

- 양의 방향으로 가장 큰 4bit 표현 : 0111
- 음의 방향으로 가장 큰 4bit 표현 : 1000
 - 여전히 최상위 자리가 부호 표시
- N-bit 2의 보수 표현의 범위
 - $[-2^{(N-1)}, 2^{(N-1)}-1]$

→ 기존 덧셈이 정상적으로 동작

$$6 + (-6)$$

$$\begin{array}{r} 111 \\ 0110 \\ + 1010 \\ \hline 10000 \end{array}$$

- 2의 보수 표현에서 부호 반전하는 방법
 1. 모든 bit를 반전 (inversion)
 2. +1

비트 폭 확장 (Increasing Bit Width)

N bit수를 M-Bit 수로 확장 ($M > N$)

- 부호 확장 (sign extension) : 최상위 비트를 복사해 상위 비트를 채움 (값을 일정하게 유지)
 - 값 3의 4bit 표현 = 0011 → 부호 확장된 8bit 표현 = 0000 0011
 - 값 -5의 4bit 표현 = 1011 → 부호 확장된 8bit 표현 = 1111 1011
- 제로 확장 (zero extension) : 상위 비트를 모두 0으로 채움 (패턴을 일정하게 유지)
 - 값 3의 4bit 표현 = 0011 → 제로 확장된 8bit 표현 = 0000 0011
 - 값 -5의 4bit 표현 = 1011 → 제로 확장된 8bit 표현 = 0000 1011 = 11(10) (원본과 달라짐)



패턴이 필요한 알고리즘일 경우 제로 확장 사용

이진 수 체계 정리

- int와 long 의 역사
 - 8-bit/16-bit 컴퓨터 시절부터 정의된 자료형 (1983년 정의된 C89)
 - C언어는 1972년 출시, 인텔 16bit 80286 (1982), 인텔 32bit 80386(1985), 인텔 64bit IA-64(2001)
 - 1983년 정의된 최초의 C89 자료형
 - char : 최소 1Byte 인 정수형
 - short(int) : 최소 2Byte인 정수형
 - int : short 보다 크고 최소 2Byte 인 정수형
 - long(int) : 최소 4Byte인 정수형
 - 64bit 구조의 등장으로 1999년 C99 표준에서 추가 정의
 - long long (int) : 8Byte 자료형

- 컴퓨터의 발전과 함께 큰 혼란이 시작됨
 - int 자료형은 가장 효율적으로 처리될 수 있는 정수타입을 목표로 정의됨
 - 컴퓨터 구조에 따라서 다른 크기였음 (16bit, 32bit)
 - 64bit 구조가 등장하면서 혼란을 막기 위해 int 자료형은 32bit 길이로 고정
 - 엄밀하게 C언어에서는 int 는 최소 2Byte 범위를 포함한 정수형으로 32bit 컴퓨터까지는 구조에 따라 크기가 달라짐, 64bit 구조 이후부터는 4Byte로 길이가 고정됨
 - 처음 정의된 long 의 입지가 불명확해짐
 - long long 은 8Byte로 명확하게 정의
 - 16bit 이하 CPU는 더이상 존재하지 않으므로 사실상 int 의 길이는 4Byte가 됨
 - 근데 long은 과거 int처럼 컴퓨터/OS 구조에 따라서 4Byte, 8Byte로 사용되고 있음에 주의
- 크로스 플랫폼 개발 시 확인 필요
 - Windows가 포함된 환경이면 가급적 long 사용 하지 않고 , int와 long long 사용 권장 (int32_t, int64_t)

고정소수점 수 체계 (Fixed-Point Number System)

- N-bit 패턴이 꼭 정수일 필요는 없음
 - 정수부(fractional part)와 소수부(integral part)를 N-bit 패턴에서 함께 표현할 수 있다
 - K-Bit 정수부, M-Bit 소수부 (K+M = N)
 - 정수부와 소수부 사이에 이진 소수점 (binary point)

$$\begin{array}{c}
 \underbrace{(x_{K-1}x_{K-2} \cdots x_1x_0)}_{\text{정수부}} \cdot \underbrace{x_{-1}x_{-2} \cdots x_{-M}}_{\text{소수부}} \\
 \\
 X = x_{K-1}2^{K-1} + x_{K-2}2^{K-2} + \cdots + x_12^1 + x_02^0 + x_{-1}2^{-1} + \cdots + x_{-M}2^{-M} \\
 \\
 = \sum_{i=-M}^{K-1} x_i 2^i
 \end{array}$$

- 이진 소수점 위치는 실제 저장하지 않음
 - 사용자(프로그래머)가 제한 없이 해당 위치를 정할 수 있음 → HW는 모두 N-bit 정수로 판단
 - 즉, 실제 의도한 값은 2^M 배 차이가 있음 (scaling factor)
 - 덧셈과 뺄셈은 이전과 동일하게 동작
 - $aX+aY = a(X+Y)$ (a= scaling factor)
 - ex) (4,4) 8bit 고정 소수점 예시 → 4bit 정수부, 4bit 실수부, scaling factor = 2^{-4}
 - $3.5 + 1.25$ 일 때? ⇒ $3.5 + 1.25 = 4.75$ (정답)
 - 3.5 를 먼저 고정소수점으로 표현한다.
 - $3.5 = a * X1$ ($a = 2^{-4}$)
 - $X1 = 3.5 / a = 3.5 / 2^{-4} = 56(10) = 0011\ 1000(2)$
 - 1.25 를 고정소수점으로 표현하기
 - $1.25 = a * Y1$ ($a = 2^{-4}$)
 - $Y1 = 1.25/a = 1.25/2^{-4} = 20(10) = 0001\ 0100(2)$
 - 실제 HW가 수행하는 연산
 - $3.5 + 1.25 = 56(10) + 20(10) = 76(10) = 0100\ 1100(2)$
 - 고정 소수점 해석으로 바꾸면

- $a(X1+Y1) = 2^{(-4)} \times 76 = 4.75$
- 곱셈과 나눗셈은 scaling factor 체크!
 - 스케일링 팩터(scaling factor), a 가 2번 적용된다.
 - $X1 * Y1 \Rightarrow (X1 * Y1 * a) * a$
- (4,4) 8bit 일 때, 고정 소수점 $\rightarrow$ (정수부 : 4, 소수부 : 4 , scaling factor : $2^{(-4)}$)
- ex) 1.5×0.5 일 때? $\rightarrow 1.5 \times 0.5 = 0.75$ (정답)
 - 1.5를 먼저 고정소수점 표현하기
 - $1.5 = a * X1$
 - $X1 = 1.5 / a = 1.5 / 2^{-4} = 24(10) = 0001\ 1000(2)$
 - 0.5를 고정 소수점 표현하기
 - $0.5 = a * Y1$
 - $Y1 = 0.5 / a = 0.5 / 2^{-4} = 8(10) = 0000\ 1000(2)$
 - 실제 HW가 수행하는 연산
 - $X1 \times Y1 = 1.5 \times 0.5 = 24 * 8 = 192$
 - $192 * a = 192 * 2^{(-4)} = 12$ (scaling factor를 한번만 적용하면 오답)
 - 한번 더 a 를 곱해야 한다.
 - $X1 * Y1 \Rightarrow X1 * Y1 * a * a$



고정 소수점의 단점

- 아주 큰 수나 작은 수를 표현할 때 너무 많은 비트를 사용한다.
 - ex) 64bit signed long long 타입(2의보수)
- $\Rightarrow$ 지수를 활용하여 큰 수, 작은 수를 더 적은 문자로 표현하자! (대신, 유효숫자의 개념을 도입)

1-2. 부동소수점 수 체계

부동 소수점 수 체계(Floating-Point Number System)

- 지수를 활용하여 수를 표기
- 숫자를 가수(M), 밑(B) , 지수(E)로 나누어 표기
 - 가수 $\times$ 밑^(지수)
 - ex) 1.23×10^3 (소수점 위치가 3칸 오른쪽)
 - ex) 2.3×10^{-5} (소수점 위치가 5칸 왼쪽)
- 소수점의 위치가 고정되지 않고, 떠다닌다~

정해진 자리 수만 사용해서 부동 소수점을 잘 표현한다

- 가수는 고정 소수점 체계를 사용
 - 다음과 같은 문제 발생
 - ex) $1234.5 \times 10^5 = 123.45 \times 10^6 = 12.345 \times 10^7 = 1.2345 \times 10^8$
 - $\rightarrow$ 모두 같다. (통일이 안됨, 정규화의 도입)
 - 10진수에서 가수는 0.0000×10^N 형태로만 표기하기로 약속 (정규화 Normalization)

- 가수의 유효숫자 문제 (rounding scheme)
 - 컴퓨터는 가수의 정해진 bit수만 사용해 저장한다.
 - ex) 1.2345×10^4 일 때, 4bit 라면, 1.235×10^4 로 표현됨. 그 방법을 정해야 한다.
 - 반올림 (round to nearest)
 - 올림 (round to toward positive infinite)
 - 내림 (round to toward negative infinite)
 - 절삭 (round to toward zero) : 소수점 이하 그냥 버림
 - 오류를 최소화 하기 위한 다양한 rounding 기법이 존재

이진 부동소수점 수 체계

- 지수적 표현을 이진수 체계에 어떻게 접목할 것인가 (컴퓨터는 2진수만 사용, 이제 2진수에서 보자.)



- 부동 소수점 수(F)는 부호(S), 지수(E), 가수 (M) 으로 표현
- $F = (-1)^S \times M \times 2^E$
- 총 비트 수 $n = 1 + e + s$, 2^n 가지 서로 다른 패턴이 있음
- 지수(E) : 보통 2의 보수 형태의 정수처럼 다룸
- 가수(M): 아래와 같이 두 가지 표현 중 선택
 - [0,1) 범위 표현 : 0.xxx ~
 - 다만, 결과적으로 앞이 0이면, 실제로 사용할 수 있는 수가 .(소수점) 뒤 밖에 안되서 1bit 손해 (결국 1로 가게 된다.)
 - [1,2) 범위 정규화 : 1.xxx ~
- 밑(base, B) : 보통 2의 거듭제곱 꼴($B = 2^k$) 로 정함
 - 2로 고정이라 쓰진 않음
- 가수가 표현 범위를 넘으면 → 가수를 줄이고, 지수를 키워 조정

ex) 5.75를 2진수로 표현하기 (부동소수점)

- 정수부 : $5 \rightarrow 101(2)$
- 소수부 : $0.75 \rightarrow 0.75 \times 2 = 1.5$
 - 정수부 1, 소수부 0.5
 - $0.5 \times 2 = 1$
 - $0.75 \rightarrow 0.11(2)$
- $5.75 = 5 + 0.75 = 101(2) + 0.11(2) = 101.11(2)$
- 정규화 하기
 - $101.11(2) \rightarrow 1.0111(2) \times 2^2$

다양한 밑(B) 옵션

- 예시: 어떤 연산의 결과가 $01.10100 \cdot 2^{100}$ 이었다면? $\rightarrow$ 사전 정의된 $[0, 1)$ 가수 범위 초과 (최대인 M_{max} 초과)
 ▶ 유효한 표현을 위한 수정작업 (correction) 필요 $\rightarrow 0.11010 \cdot 2^{101}$
- ▶ 밑이 2가 아니면? $\beta = 2^k$ 상황에서는 지수를 1 증가시키기 위해선 가수를 k 위치 이동함
 $\rightarrow \beta = 4 = 2^2$ 상황에서 수정된 유효한 표현 = $01.10100 \cdot 2^{100} = 01.10100 \cdot 4^{010} = 00.01101 \cdot 4^{011}$

해당 장표에 나온 숫자 $01.10100 \times 2(100)$ 에서 가수 부분도 이진수, 2의 지수인 100도 이진수 입니다.

계산 결과가 $01.10100 \times 2^{(100(2))}$ 였다면?

$\rightarrow$ 정규화를 통해 $0.11010 \times 2^{(100(2))}$ 로 변경

$\rightarrow$ 밑을 2진수로 표기하니 너무 표기가 커서, 2의 거듭제곱 꼴로 표현 가능

$$01.10100 \cdot 2^{100} = 01.10100 \cdot 4^{010} = 00.01101 \cdot 4^{011}$$

$$\textcircled{1} \quad 2^{100(2)} = 2^{4(10)} = 4^2 = 4^{010(2)}$$

$$\textcircled{2} \quad 01.10100 \cdot 4^{100(2)} = 00.01101 \cdot 4^{011(2)}$$

밑이 2일 때는 2진수에서 1칸씩 이동하지만, 밑이 4가 되어, 2칸씩 이동 가능

밑이 2일 때는 2진수에서 소수점을 한 칸씩 이동 가능하지만, 밑을 2의 거듭제곱 꼴인 4로 표현하면, 두 칸씩 이동이 가능하다!



다만! 밑이 커질 경우, 정밀도가 떨어지는 단점이 있다

$\rightarrow$ 컴퓨터 과학/공학 계산에서는 넓은 범위의 표현보다 정밀도가 더 중요하다(이진 부동소수점 체계에서는 밑을 2로만 표현)

정규화의 장점

- 가수 부분의 포맷을 일정하게 유지
 - 결과적으로 $1.xxxx \times 2^E$ 로 고정
 - $0.xxxx$ 은 맨 앞이 0이어서, 1bit 표현을 손해 보게 됨
- 지수 부분만 비교하면, 바로 두 수의 크기 비교가 가능
 - 지수 비교 $\rightarrow$ 가수 비교 순으로 진행

정규화 표현의 한계

- 정규화된 표현은 0을 포함하지 않음
- 0을 표현하기 위한 특수 패턴을 정의
 - 가장 일반적인 표현 : $M=0, (E=0 \text{ 선택})$

바이어스/편향 지수(biased exponent)

- 부동 소수점에서 지수가 저장되는 방식 (저장되는 지수는 실제 지수가 아니다)
- 만약 실제 지수(Etrue)가 음수일 수도 있음
 - ex) 2^{-3}
- 매 번 음수인 지수를 양수로 변환해서 저장하는 것은 매우 비효율적

→ Bias 를 도입해서 지수에 더해서 양수로 만들어서 항상 양수로 저장하자!

→ $E = E_{true} + bias$ 로 사용

Bias 소개

Bias는 미리 정해진 상수

- $Bias = 2^{(e-1)}$ (여기서 e 는 지수 비트 수, DEC 포맷(옛날 방식, 요즘은 $2^{(e-1)} - 1$ 로 한다. IEEE 표준)
 - ex) 8bit 지수 → $bias = 2^{(8-1)} = 2^7 = 128$
 - ex) 11bit 지수 → $bias = 2^{(11-1)} = 2^{10} = 1024$
 - ex) 8bit 지수, 2^{-3} 이 있을 때, ($e = 8$)
 - $bias = 2^{(8-1)} = 128$
 - $E_{true} = -3$
 - $E = E_{true} + bias = -3 + 128 = 125$
 - 메모리에 저장되는 지수
 - $125 = 0111\ 1101(2)$

e-bit Etrue 의 표현 범위

- $-2^{(e-1)} \leq E_{true} \leq 2^{(e-1)} - 1$
 - ex) $e = 3$ 일 때,
 - $-2^{(3-1)} = -4$
 - $2^{(3-1)} - 1 = 3$
 - $-4 \leq E_{true} \leq 3$
- 바이어스는 일반적으로 지수가 가질 수 있는 가장 작은 음수의 절대값으로 설정 ($bias = 2^{(e-1)}$)
 - excess $2^{(e-1)}$ 표기법 → $0 \leq E \leq 2^e - 1$
 - ex) $e = 7$ 이면, (지수 비트수 7)
 - $2^{(7-1)} = 64$, excess 64 표기법
 - 실제 지수의 범위 : $-2^7 \leq E_{true} \leq 2^7 - 1 \Rightarrow -64 (= 000\ 0000(2)) \leq E_{true} \leq 63 (= 011\ 1111(2))$
 - $bias = 64$
 - $E_{true} = -64$ 이면, $E = E_{true} + bias \Rightarrow -64 + 64 = 0 = 000\ 0000(2)$ 기록
 - $E_{true} = 63$ 이면, $E = E_{true} + bias \Rightarrow 63 + 64 = 127 = 111\ 1111(2)$ 기록
- excess $2^{(e-1)}$ 기법을 사용하면, 실제 지수의 2의 보수표현에서 부호 비트만 반전시키면 됨



2의 보수와 excess $2^{(e-1)}$ 표기가 사실 연결되어 있다!

- 바이어스 지수의 장점
 - 지수를 0/양수 영역으로 이동 시켜 기록된 지수를 부호없는 수 표현으로 해석하게 만들

- 정규화 된 가수와 바이어스 지수
 - S,E,M 표현 방식의 대소 비교가 부호 없는 정수 표현의 경우와 동일해짐

이진 부동 소수점의 표현범위

- 부동 소수점 수에서 일반 표현식
 - $F = (-1)^S \times M \times B^E$
 - 일 때, 양수/음수 영역을 따로 생각할 수 있다.
 - 양수 : $+F$ 의 범위
 - 음수 : $-F$ 의 범위
- 양수 영역만 고려했을 때, F의 범위는 다음과 같다.
 - $M_{min} \times B^{(E_{min})} \leq F \leq M_{max} \times B^{(E_{max})}$
 - 지수가 E_{max} 초과 하면, overflow(→ 무한대로 표시, 또는 인터럽트 발생 or (비선호방식) 표현 가능한 최대값으로 표현)
 - 지수가 E_{min} 미달이면 underflow(→ 0으로 표시)

→ 결과적으로 정규화된 표현 범위에서 0이 수 체계에 포함되지 않는다.

부동 소수점 표준 (IEEE 754 Standard)

- 컴퓨터 시스템에서 가장 널리 사용되는 이진 부동 소수점 표준
 - Single-precision 32-bit 포맷 (c언어에서 float 자료형)
 - Double-precision 64-bit 포맷 (c언어에서 double 자료형)
 - Single-extended 포맷 (최소 44bit 사용)
 - Double-extended 포맷 (최소 80bit 사용)
 - 중간값 계산 과정에서 활용되며 보다 높은 해상도/범위 제공
 - 유효숫자 제한으로 인한 오류를 줄이는 rounding scheme 들
 - Round to nearest even (mandatory)
 - Round toward zero
 - Round toward positive/negative infinite

IEEE Single-Precision 포맷

S	8 bits - biased exponent E	23 bits - unsigned fraction f
-----	------------------------------	---------------------------------

S(1bit) : 부호

E(8bit) : 편향 지수

f(Fraction, 23bit) : 가수부에서 소수점 뒤 부분

- 공식

$$F = (-1)^S \times 1.f \times 2^{(E-127)}$$

- 특징
 - 밑(B) : 2 고정
 - Hidden bit 사용 : 1.xxxx 형태로 가수를 정규화
 - 맨 앞의 1을 f에 저장하지 않음 (맨 앞이 1이라 생략, 숨겨진 1(Hidden bit) 로 취급)
 - 그래서 23bit만 저장하지만, 실제 정밀도는 24bit의 효과를 냄
 - 8-bit 지수에 bias를 127로 사용 (256가지 지수 표현 중 특수 패턴 정의)

- $E=0$
 - $f=0 \rightarrow 0$ (+0 or -0)
 - $f \neq 0 \rightarrow$ 비정규화 수 의미
- $E = 255$
 - $f=0 \rightarrow$ +-무한대(infinity)
 - $f \neq 0 \rightarrow$ NaN (Not A Number)
- $1 \leq E \leq 254$
 - $F = (-1)^S \times 1.f \times 2^{(E-127)}$
 - 일반적인 정규화 수

조금이라도 넓은 표현 범위를 위해

- IEEE : 지수 bias : 128 대신 127 사용
 - 실제 지수의 최대 값 증가 : $E_{true\ max} = 254 - 127 = 127$
- 가수에 Hidden bit 사용
 - 0.xxx $\rightarrow$ 1.xxx 사용
 → 지수에 1 더하는 효과
- 양의 방향 범위
 - $F_{max} = (1 - 2^{-24}) \times 128$
 - DEC 포맷에서는 $F_{max} = (1 - 2^{-24}) \times 2^{127}$
 - IEEEC 표준의 경우 모든 정규화된 수의 역수를 상한 초과 없이 표현할 수 있다.

1-3. 연산과 복잡도

2. 모델 경량화와 효율적 파인 튜닝

2-1. 모델 경량화의 필요성

AI/ML 모델의 성능이 우수해 지고 있다.

- 모델의 파라미터의 크기가 너무 크다.
- 컴퓨팅 리소스 문제가 발생
- 학습 비용이 많이 든다.

모델은 계속해서 커질 예정

- 모델의 경량화는 선택이 아닌 필수

일반적으로 경량화의 정도가 좋아지면, 모델 성능이 떨어짐

→ 어떻게 성능을 개선할 것인가

GPU와 궁합이 좋은 경량화 기법

- LoRA(Low-rank adaption), QLoRA (Quantized low-rank adaption)
- Knowledge distillation
- Constrained quantization
- Structed pruning

GPU와 궁합이 좋지 않은 기법 (전용 가속기 필요)

- Aggressive quantization, mixed-precision quantization
- Unstructured/ partially-structured pruning

2-2. Quantization/Pruning/Distillation

양자화 (Quantization)

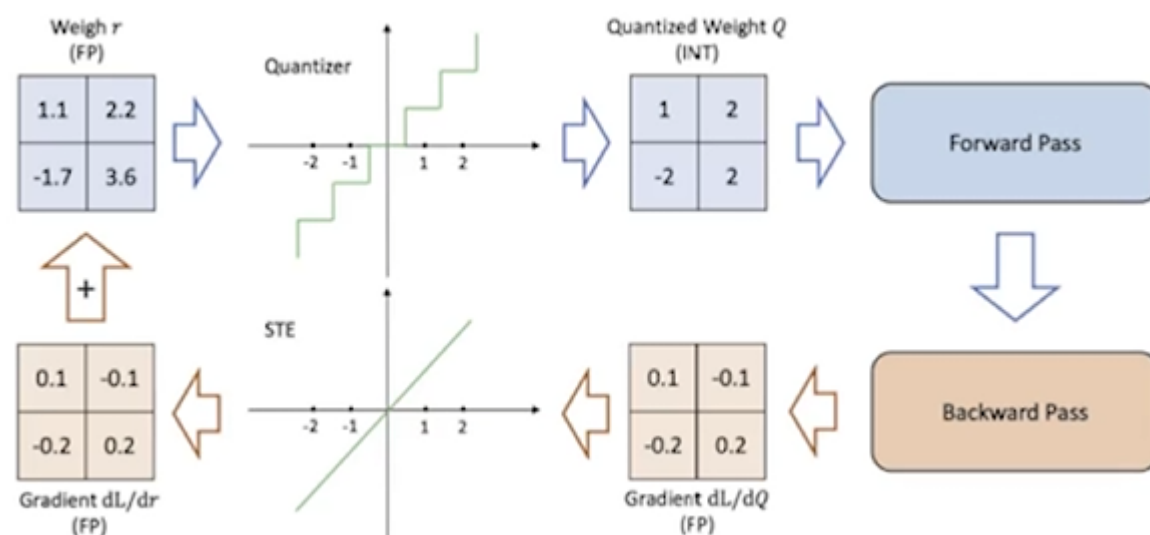
- 연산/메모리 부하를 줄이는 가장 직관적인 방법
 - Clipping range에 따른 scaling factor 정하는 것이 가장 첫 단계
 - 모델마다 적용 가능한 양자화 옵션이 다른 경우가 많음
 - 효율 개선을 위해 하드웨어/가속기 지원이 필요한 경우도 있

양자화 종류

- QAT , PTQ 로 구현 가능



- QAT
 - Quantization-Aware Training (QAT)
 - 학습 단계에서 (forward pass) quantization 을 함께 수행 (학습 데이터가 필요)
 - 동일 accuracy 기준 압축률을 높일 수 있음, but 학습 복잡도가 엄청 상승 (retrain / fine-tuning 접근)
 - 상대적으로 CNN 등 오리지널 학습 복잡도/데이터가 크지 않은 경우 쉽게 적용 가능

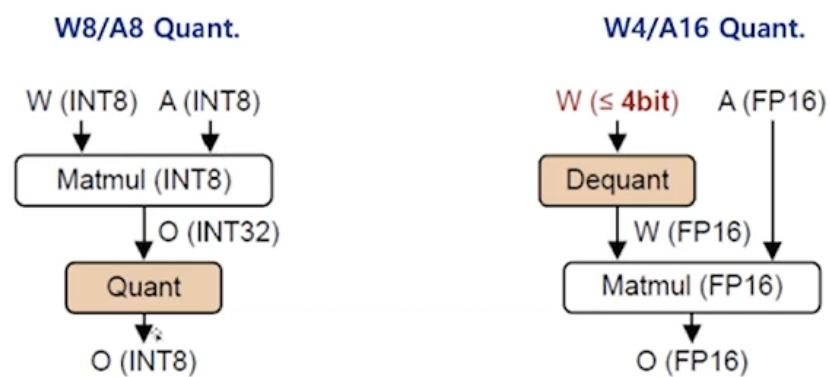


- PTQ

- Post-Training Quantization (PTQ)
- LLM 처럼 모델도 학습 데이터도 너무 큰 경우엔 QAT 불가
- 사전 학습 된 모델을 바탕으로 양자화 바로 수행
- 적절한 calibration 데이터를 사용해서 경량화 효율을 높일 수 있음

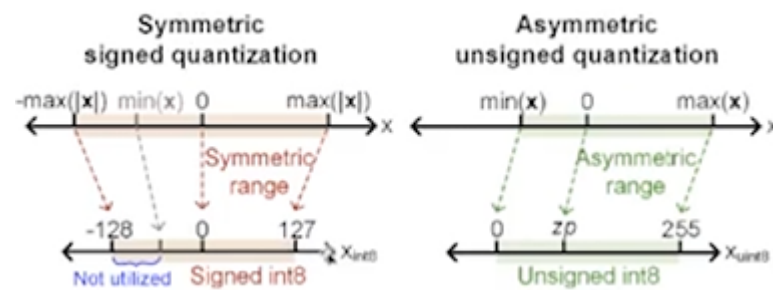
PTQ의 방법

1. Weight-activation(둘 다 양자화), Weight-only(Weight만 양자화)



- 모델 사이즈에 얼마나 집중할 것인가? (정확도 손실을 어디까지 허용할 것인가?)
- De-quantization 과정을 얼마나 허락할 것인가?
- Weight : 가중치
- Activatoin : 중간 계산값
- Weight-activation(둘다 양자화)
 - FP32 → INT8
 - 추론 과정 전체에서 메모리 접근량과 연산량 크게 줄임
 - 연산은 정수 처리되어 하드웨어 가속기에서 효율적
 - 장점
 - 모델 크기, 연산 속도 둘다 줄이기 가능
 - 모바일, 엣지 디바이스에 많이 사용
 - 단점
 - 연산 값이 줄어들어 정확도 손실이 커진다.
 - 쓸모없는 모델이 될 수 있음
- Weight-only
 - 가중치만 정수로 변환 / 활성값은 그대로
 - 요즘 모델은 파라미터도 많고 용량이 커서 용량 줄이는 게 급선무일 경우 사용
 - 장점
 - 정확도 손실이 Weight-activation 보다 적음
 - 학습된 모델을 큰 수정 없이 적용하기 쉬움
 - 단점
 - 속도 이득이 제한적
 - 메모리 이득은 있지만, 하드웨어 최적화 효과는 덜

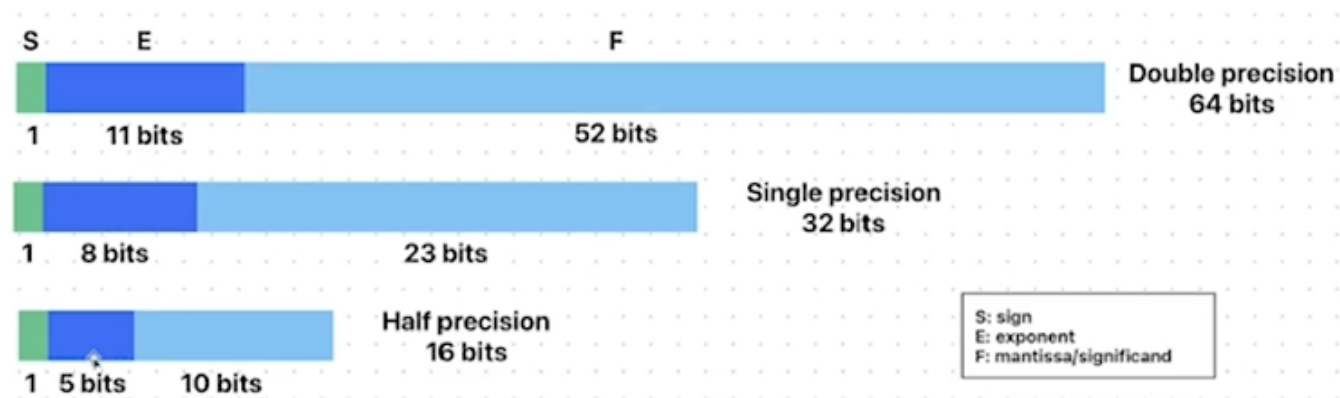
2. Symmetric VS Asymmetric



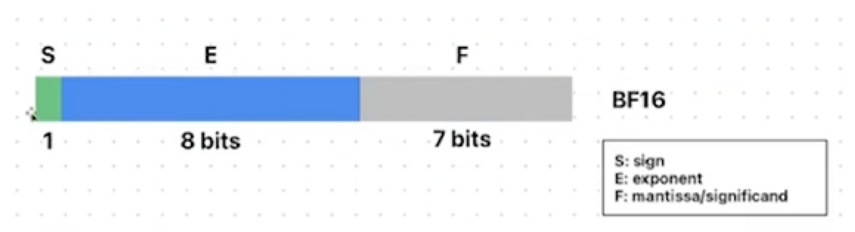
- Activation 및 weight의 분포를 어디까지 반영할 것인가?
- 하드웨어 연산기 수준의 지원은?
- 대칭 양자화 (Symmetric) : 양자화 구간이 0을 중심으로 대칭적으로 설정
 - 최소값 = - 최대값
 - scale만 사용
 - 장점
 - 연산이 단순
 - 행렬 곱셈에 적합 (GPU/TPU 가속기에서 효율적)
 - 단점
 - 데이터 분포가 비대칭일 때 표현 낭비가 발생
- 비대칭 양자화 (Asymmetric) : 양자화 구간이 0을 중심으로 하지 않아도 됨
 - scale + zero-point 둘 다 사용
 - 장점
 - 데이터 분포가 0에 치우치지 않아도 효율적 표현 가능
 - 특이 활성화 값 양자화에 유리
 - 단점
 - 연산이 복잡 (scale, zero-point 적용)
 - 하드웨어 가속기에서 덜 효율적

3. Integer VS Floating

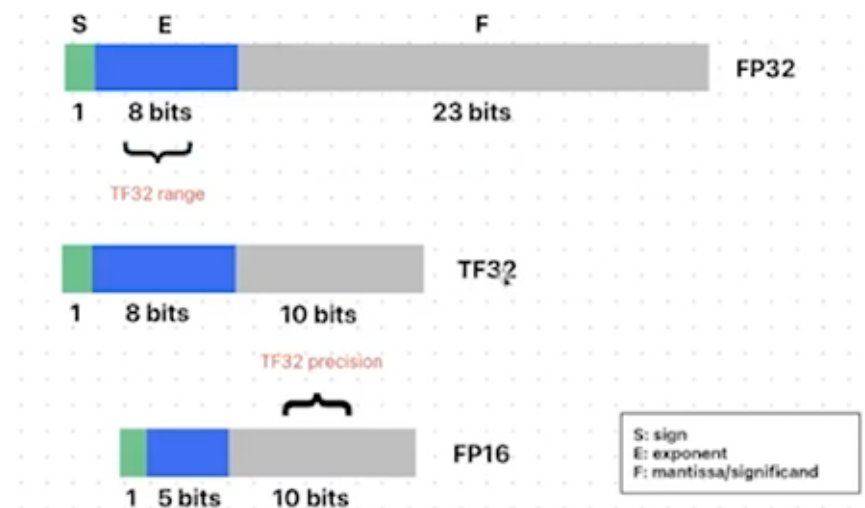
- Floating 에는 다양한 방법이 있다.
 - 기존 IEEE 754 표준 사용



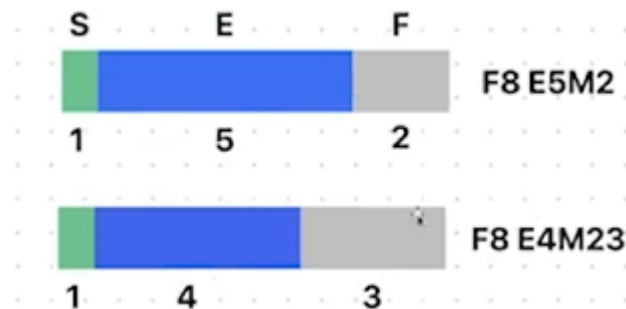
- 구글 BF16 표현 (IEEE 의 FP32의 truncation 버전)



- TensorFloat-32 (TF32) 표현 (IEEE FP32의 범위, FP16의 유효숫자)



- FP8 (E5M2 vs E4M3)



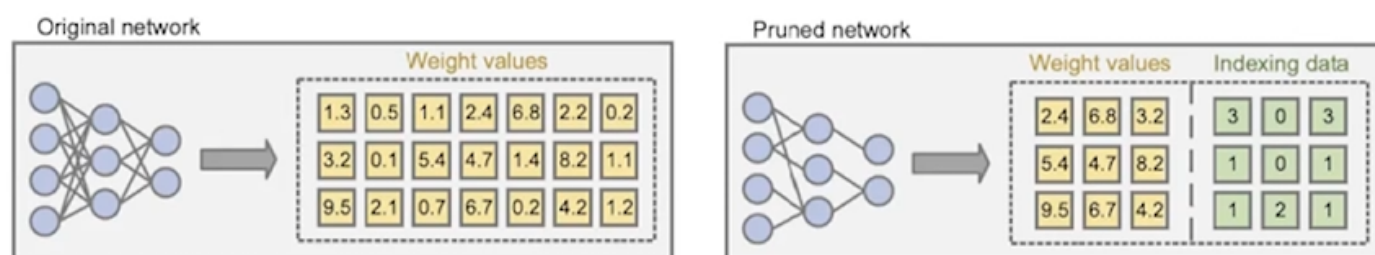
💡 다양한 양자화 기법에 대해 논쟁이 있다.

Mixed-precision Quantization

- 레이어 별로 최적화 된 양자화 적용

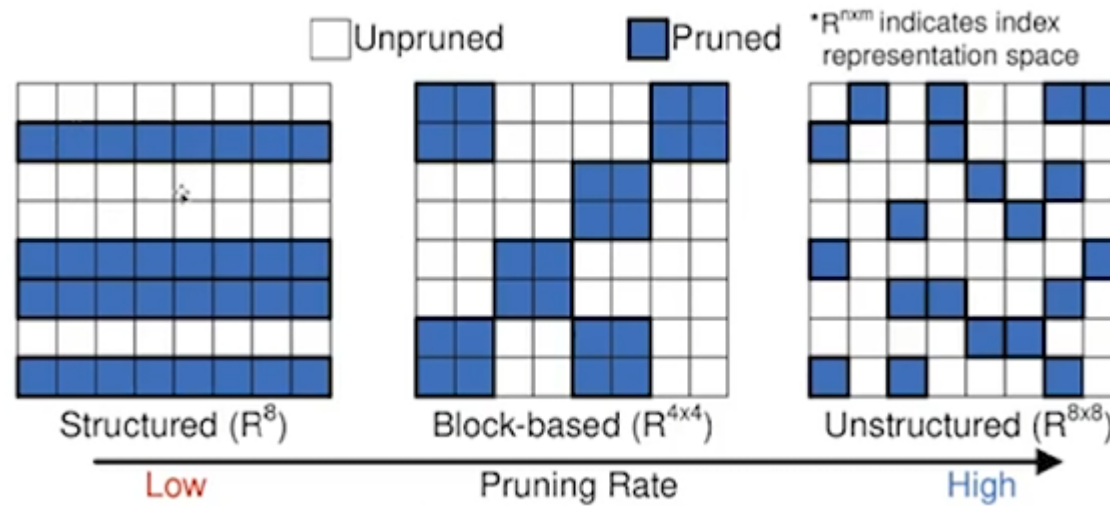
Pruning (가지치기)

- 불필요한 Weight 제거
 - 저장해야 하는 weight 가 적어지면 모델의 크기가 줄어든다.
 - 살아남은 weight의 위치를 기록하기 위한 추가 정보 필요 (indexing data, meta data 등)
 - 양자화가 더이상 되지 않는 상에서도 pruning 이 가능
 - GPU에서는 속도 이득을 보기 어려울 수 있다.



- Pruning 방법 (가지치기 대상의 순서를 정하기)
 - magnitude-based pruning (가장 쉬운 방법)
 - Pruning ratio (압축률) 대비 높은 정확도를 유지하는 다양한 기법 존재 (momentum-based pruning)
- Unstructured(패턴 없게) VS Structured(패턴 있게)
 - 이 두 방법만 이해해도 된다.
 - 추론(inference) 효율과 accuracy 방어 사이의 선택

- Channel/layer 수준의 극단적인 structured pruning 도 활용 가능



Indexing data format

- 가장 일반적으로 쓰는 CSR (Compressed sparse row) → 하지만 pruning ratio 가 극단적으로 높아야 GPU 가속
 - CSR : 엔비디아에서 제안
- on-off encoding, XOR-gate compression 등 새로운 시도가 있다.

Fine-tuning / retraining 가능 여부

- 양자화의 경우와 비슷한 양상
 - 작은 모델, 학습 데이터에서는 training 단계부터 도입 가능 (CNN 등)
 - LLM 같이 큰 경우에는 사후학습 pruning이 현실적
- 모델 구조에 따라 상이하지만, CNN기반 이미지처리는 80% 이상도 pruning 가능 (1~2% 정확도 손실)
- 언어 처리를 위한 LLM 계열은 50% 내외 수준

GPU 호환 여부

- 양자화와 비슷하게 GPU가 감당할 수 없는 pruning 패턴이 형성되는 경우가 있다.
- AI/ML 모델 특성상 기본적으로 0이 많이 포함
- 전용 가속기 및 NPU 설계단계에서 sparsity-aware 구조를 많이 고민하고 있

지식 증류 (Knowledge Distillation)

- 작은 모델을 만드는 효과적인 방법
 - 똑똑하지만 매우 큰 모델에서 출발 (Teacher)
 - 동일한 문제를 푸는 작은 모델(student)의 학습에서 선생의 역할을 함
 - Edge에 적용 가능한 매우 쓸만한 작은 모델 제작 가
- 방법
 - 두가지 loss 정의 : Total loss = student loss + distillation loss
 - student loss : Ground truth의 student 추론의 차이
 - distillation loss : 동일 데이터에 대해서 teacher와 student의 결과 차
- 지식 전달/증류의 효과적인 방법
 - 조교를 활용하기 (Assistant model)
 - teacher 모델과 student 모델의 간극이 클 때 중
 - 조교의 효과

- NOKD : 학생을 바로 학습 (NO Knowledge Distillation)
- BLKD : 선생님 혼자 학생 교육 (Baseline Knowledge Distillation)
- TAKD : 조교의 개입 (Teacher Assistant Knowledge Distillation)

LLM Distillation

- 상대적으로 작은 LLM 학습에 기존 LLM 사용
 - 성능이 매우 뛰어남
- 다만, 학습 데이터가 너무 크면 여전히 문제
 - 합성 데이터 활용 : 프롬프트 데이터로 선생님 모델의 응답을 뽑고 이것을 학생 모델이 fine tuning

2-3. LoRA / QLoRA

파인 튜닝

- 모델 경량화 관점에서 파인 튜닝
 - 원본 모델의 크기를 줄이는 과정
 - 일부/전체 학습 데이터를 바탕으로 떨어진 정확도를 복원하는 과정
 - QAT 등
- 모델 서비스 관점에서 파인 튜닝
 - 일반적인 LLM 모델을 새로운 도메인에 특화시키자 한다면?
 - 조정/튜닝 하여 서비스

다양한 파인 튜닝 방법론이 존재

1. In-Context 학습

- 원본 모델을 두고 원하는 방향으로의 동작을 튜닝하기 위한 프롬프트 고민

2. Full fine tuning

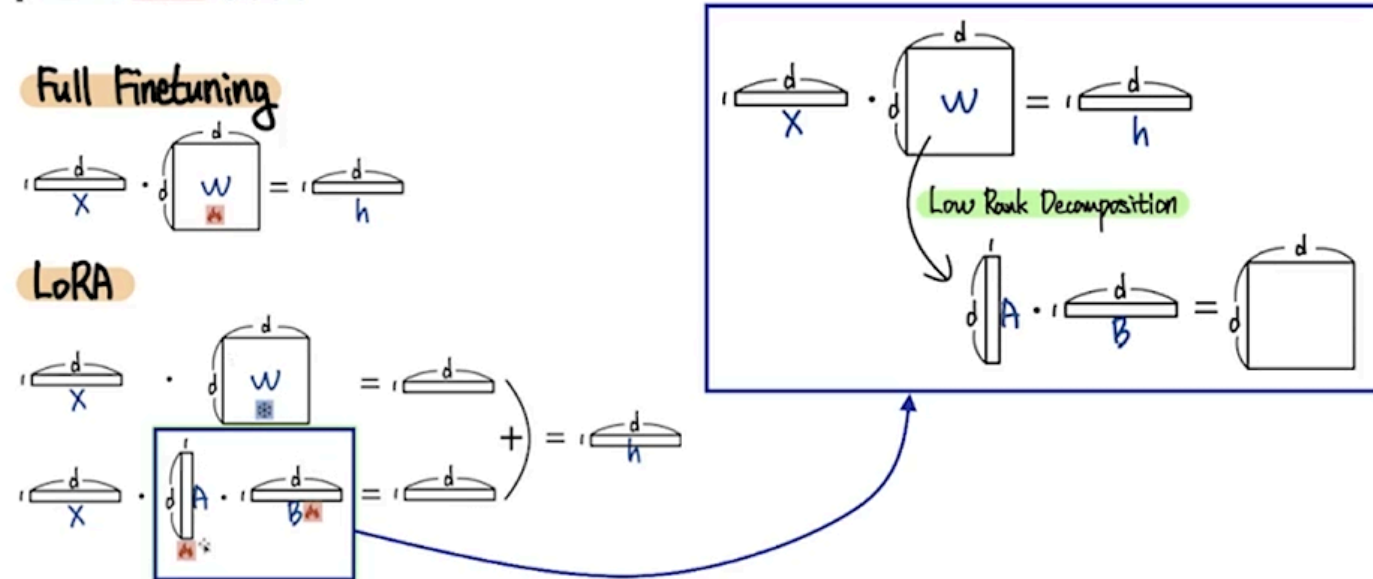
- 특정 도메인의 데이터를 사용하여 원본 모델의 모든 weight를 finetuning
- 장점
 - 처음부터 새로 학습하는 것에 비해 적은 데이터만 활용해도 도메인 특화 가능
 - 도메인 특화 관점에서 정확도 이득 기대 가능
- 단점
 - 원본 모델의 모든 weight를 갱신하는 학습 비용 부담
 - 메모리 사용량 관점에서 처음부터 모델 생성하는 수준일 수 있음
 - 원본 모델의 weight가 변화하여 기존 학습 정보의 망각이 발생할 수 있음

파라미터 효율적 파인튜닝

- LLM을 도메인 특화 시키는 과정의 복잡도 개선
 - 원본 모델의 전체 weight를 전부 갱신하지 않는 튜닝 방법 지원
- 기본적으로 모델을 특화시키기 위한 추가 weight를 어디에/어떻게 추가할 지에 대한 문제
- Adapter Layer
 - 입력 원본 모델 내부에 새로운 레이어 추가 (MLP 형태)
 - 목표로 하는 특화 데이터를 활용 해 adapter layer만 학습 (원본 weight 고정)

- 장점
 - full fine tuning 보다는 적지만 상당한 추가 weight 필요, 대신 꽤 높은 표현력 제공
- 단점
 - 원본 모델의 구조가 바뀌고, adapter layer의 메모리/연산 오버헤드가 크다
- Prompt tuning
 - 입력 임베딩 앞/중간에 학습 가능한 가짜 토큰 (pseudo-token) 추가
 - 원본 모델의 구조를 전혀 건들이지 않고, 아주 적은 추가 weight로 입력을 조정하여 특정 동작을 유도
 - 매우 가볍고 배포가 쉬운 장점이 있으나, 표현력의 한계로 성능에 제한
- Low-Rank Adaptation(LoRA)
 - 특정 weight matrix (base weight)에 low-rank update를 추가
 - 원본 weight를 고정하고, 추가되는 low-rank weight만 학습
 - 높은 표현력에 adapter layer보다 훨씬 적은 추가 weight

! 그림으로 LoRA 이해하기



QLoRA

- 양자화 + LoRA
- 원본 모델을 강하게 양자화 (ex) 4bit normal format, NF4) → 범용 성능의 하락을 피할 수 없
- LoRA를 적용하는 low-rank weight는 BF16등 높은 해상도를 유지하며 학습
- 전체 모델의 복잡도를 대폭 줄이면서 목표하는 task에 대해서는 높은 성능 달성

실습

- LoRA/ QLoRA 활용하기
- LoRA/QLoRA 적용을 위한 최신 파인튜닝 라이브러리 활용 (Unsloth, Axolotl)
 - LoRA/QLoRA 구현상 요구되는 파라미터 이해 (rank, alpha, target_modules)
 - 선택한 데이터에 특화된 파인튜닝 과정 전반을 경험

주요 실습 포인트

- Base 모델 및 토크나이저 로딩
- LoRA 설정 및 적용
- 데이터셋 준비 및 전처리
- 모델 학습 및 저장
- 파인튜닝된 모델로 추